



Corso: Data Analyst

Studentessa: Carol Pagano

Data Consegna: 08-06-2026

Oggetto: Progettazione di basi dati

Risposte alle Domande Teoriche

(dal file "rev1" rinominato "Traccia Esercizio Finale M2_W8")

1. Cosa si intende per database?

Un database (o banca dati) non è solo un archivio ma è un sistema organizzato progettato per memorizzare dati in modo logico, coerente e ordinato, permettendo di gestire sia il contenuto che le relazioni tra i dati stessi (modello E-R= Entity-Relationship). L'obiettivo principale è evitare errori, ridondanze e ripetizioni.



2. Cos'è un DBMS?

Un DBMS (Database Management System) è il software di sistema che funge da interfaccia tra il database fisico e gli utenti o le applicazioni. Permette di creare, scorrere, aggiornare e amministrare la base dati. MySQL Workbench è un classico esempio di client che si interfaccia con un DBMS (MySQL). Il suo compito fondamentale è garantire la definizione delle strutture dati (DDL), la manipolazione dei record (DML) e il controllo degli accessi (DCL). Un DBMS di livello professionale garantisce la consistenza transazionale del sistema attraverso il rispetto delle proprietà A.C.I.D. (Atomicity, Consistency, Isolation, Durability), gestendo la concorrenza degli accessi simultanei senza conflitti.



3. Indica le principali clausole di uno statement SELECT in ordine di esecuzione logica. Descrivi per ciascuna delle clausole indicate la logica di funzionamento.

Il motore di esecuzione SQL valuta le clausole di una query in un ordine logico differente rispetto alla sintassi di scrittura. L'ordine di esecuzione logica è il seguente:

1. **FROM (+ eventuali JOIN):** indica la tabella di origine da cui estrarre i dati.
Risponde alla domanda: *Dove si trovano?*
2. **WHERE:** applica un filtro riga per riga. Solo i record che soddisfano la condizione (risultano TRUE) vengono estratti.



Risponde alla domanda: *Quali righe scelgo?*

3. **GROUP BY:** trasforma molte righe in blocchi logici (gruppi) basati su valori identici.

Risponde alla domanda: *Come li raggruppo?*

4. **HAVING:** Applica un filtro condizionale sui dati aggregati dal GROUP BY (lavora sulle funzioni di aggregazione come SUM, COUNT, AVG).

Risponde alla domanda: *Quali blocchi tengo?*

5. **SELECT (+ DISTINCT):** estrae le colonne specificate, calcola le espressioni e assegna gli alias. Se è presente la clausola DISTINCT, in questa stessa fase vengono eliminate le righe duplicate dal result set prodotto – DISTINCT non è una fase separata, ma parte integrante dell'elaborazione SELECT.

Risponde alla domanda: *Cosa voglio vedere?*

6. **ORDER BY:** Ordina il result set finale in base ai criteri specificati (ASC/DESC). In MySQL, ORDER BY è la clausola che più comunemente utilizza gli alias di colonna, ma anche HAVING (e in alcuni casi GROUP BY) può riferirsi agli alias definiti nella SELECT.

Risponde alla domanda: *Come li ordino?*

7. **LIMIT:** Limita il numero di righe restituite (fondamentale per Dataset enormi).

Risponde alla domanda: *Quanti ne vedo?*



4. Descrivi, immaginando uno scenario a te familiare, il concetto di group by. Utilizza l'approccio che ritieni più efficiente per trasmettere il concetto (suggerimento: disegna anche una sola tabella in Excel o in word con poche colonne e pochi record e descrivi, basandosi sulla tabella stessa, un esempio di group by).

Per comprendere il funzionamento della clausola GROUP BY, immaginiamo di analizzare il registro delle vendite giornaliere di un punto vendita Legami Milano. La tabella sottostante rappresenta il nostro set di dati di partenza (composto da 10 record), contenente informazioni sui prodotti venduti, la loro categoria merceologica di appartenenza, il prezzo unitario e la quantità acquistata dai clienti.

Tabella di Partenza:

ID_Transazione	Nome_Prodotto	Categoria	Prezzo_Unitario (€)	Quantità_Venduta
1	Penna Gel Cancellabile Orsetto	Penne	2.50	12
2	Taccuino Medium Righe Flora	Cartoleria	9.90	5
3	Penna Gel Cancellabile Tratto	Penne	2.50	20
4	Evidenziatore Profumato Teddy	Evidenziatori	1.90	15
5	Taccuino Pocket Quadretti	Cartoleria	5.90	8



6	Astuccio Soft Touch Nero	Accessori	12.90	3
7	Penna Gel Cancellabile Koala	Penne	2.50	18
8	Evidenziatore Pastel Rosa	Evidenziatori	1.90	10
9	Diario Giornaliero Panda	Cartoleria	14.90	4
10	Zaino Foldable Tech	Accessori	24.90	2

Se la direzione aziendale chiedesse di monitorare le performance di vendita non più sul singolo articolo, ma per macro-categoria, la clausola GROUP BY Categoria diventerebbe lo strumento fondamentale.

Risultato del Raggruppamento

Dopo aver applicato il GROUP BY Categoria, il nostro result-set si trasforma in una sintesi strategica per il business:

Categoria	Totale_Pezzi_Venduti	Fatturato_Totale (€)
Penne	50 (12 + 20 + 18)	125.00
Cartoleria	17 (5 + 8 + 4)	156.30
Evidenziatori	25 (15 + 10)	47.50
Accessori	5 (3 + 2)	88.50



5. Differenza tra schema OLTP e schema OLAP.

- **OLTP (Online Transaction Processing):** sono sistemi focalizzati sull'operatività quotidiana (es. transazioni bancarie). Devono essere veloci e garantire che ogni operazione sia consistente e sicura. Lo schema è fortemente normalizzato (3NF) per evitare anomalie e garantire l'integrità dei dati operativi in tempo reale. Quindi questo è → l'approccio Transazionale.
- **OLAP (Online Analytical Processing):** qui i dati storici vengono riorganizzati per supportare decisioni strategiche tramite il Data Warehouse (arriva dopo il caricamento dei dati e serve per l'analisi). Utilizza processi ETL per organizzare i dati a fini decisionali. Quindi questo è → l'approccio Analitico.



6. Dato un medesimo scenario di analisi, qual è la differenza in termini di risultato ottenibile tra una join e una subquery?

A parità di scenario, la differenza principale risiede nella struttura del result set finale e nella flessibilità d'uso delle colonne:



- **JOIN:** Unisce fisicamente le tabelle orizzontalmente. Questo permette di includere nella SELECT finale le colonne provenienti da entrambe le tabelle coinvolte nella relazione.
- **Subquery:** Restituisce un set di dati temporaneo utilizzato come filtro o termine di confronto dalla query principale. Se usata nel WHERE, ad esempio, permette di filtrare i record della tabella principale in base a dati di una seconda tabella, ma non permette di mostrare direttamente nel risultato finale le colonne di quest'ultima.



7. Cosa si intende per DML e DDL?

L'SQL funziona come un ponte: noi inviamo un'istruzione al database, lui la legge, la elabora e ci restituisce un output.

Le istruzioni SQL si dividono in 3 macro-categorie:

- **DDL (*Data Definition Language*):** include i comandi utilizzati per definire, modificare o eliminare la struttura degli oggetti del database (es. CREATE per creare nuove strutture, ALTER per modificarle, DROP per eliminarle). Non agisce sui dati, ma sul contenitore.
- **DML (*Data Manipulation Language*):** include i comandi utilizzati per manipolare e gestire i dati all'interno delle strutture esistenti (es. SELECT per visualizzare, INSERT per inserire, UPDATE per modificare, DELETE per eliminare). Non agisce sulla struttura, ma sui dati stessi.
- **DCL (*Data Control Language*):** (*introdotta come approfondimento*) include i comandi utilizzati per gestire la sicurezza e i permessi di accesso al database (es. GRANT per concedere autorizzazioni, REVOKE per revocarle). Non agisce su strutture o dati, ma sui diritti degli utenti.



8. Quali istruzioni possono utilizzare per estrarre l'anno da un campo data? Proponi degli esempi.

In MySQL esistono più istruzioni per estrarre componenti temporali da un campo data. I tipi di campo su cui queste funzioni operano sono **DATE**, **DATETIME** e **TIMESTAMP**.

1. Funzione YEAR() — Estrae il valore dell'anno come numero intero a 4 cifre. È la funzione nativa MySQL più utilizzata per questo scopo:

```
SELECT YEAR(SalesDate) AS AnnoDiRiferimento FROM Sales;
```

Una vendita con SalesDate = '2025-06-25' → Restituisce: 2025

2. Funzione MONTH() — Appartiene alla stessa famiglia di YEAR() e ne segue la medesima logica sintattica, ma estrae il mese come numero intero (1-12):

```
SELECT MONTH(SalesDate) AS MeseDiRiferimento FROM Sales;
```



Una vendita con SalesDate = '2025-06-25' → Restituisce: 6

Allo stesso modo, DAY() estrae il giorno. Le tre funzioni YEAR(), MONTH(), DAY() formano la famiglia standard MySQL di estrazione delle componenti temporali.

3. Funzione EXTRACT() — *(introdotta come approfondimento)* Sintassi SQL standard, compatibile con più DBMS. Produce lo stesso risultato di YEAR() ma con una struttura più esplicita:

```
SELECT EXTRACT(YEAR FROM SalesDate) AS AnnoDiRiferimento FROM Sales;
```

Una vendita con SalesDate = '2025-06-25' → Restituisce: 2025

La stessa funzione può estrarre anche MONTH, DAY, HOUR ecc. semplicemente cambiando il parametro.

Nel contesto di questo progetto, all'interno delle query di analisi del database **ToysGroup**, YEAR() è stata la scelta adottata per la sua chiarezza e la piena compatibilità con la sintassi nativa MySQL Workbench.



9. Differenza tra gli operatori logici AND e OR

- **AND:** Restituisce TRUE solo se tutte le condizioni collegate sono contemporaneamente vere. Filtra i dati in modo restrittivo.

Esempio: **WHERE Category = 'Dolls' AND Price > 20.00** estrae solo i giocattoli Dolls con prezzo superiore a 20€.

- **OR:** Restituisce TRUE se almeno una delle condizioni collegate è vera. Estende la ricerca includendo più casistiche.

Esempio: **WHERE Category = 'Dolls' OR Category = 'Toys'** estrae tutti i giocattoli appartenenti a Dolls oppure a Toys.



10. È possibile innestare una query nella clausola SELECT?

Sì, è possibile. Viene chiamata **Subquery Scalare**. Affinché funzioni correttamente, la subquery deve restituire rigorosamente un solo valore (una sola riga e una sola colonna) per ogni riga della query principale, altrimenti il sistema restituirà un errore di esecuzione.

Esempio su ToysGroup: **SELECT SalesCode, Price, (SELECT AVG(Price) FROM Sales) AS MediaGlobale FROM Sales** — la subquery scalare restituisce un unico valore (la media globale dei prezzi) che appare identico in ogni riga del result set.

11. Differenza tra l'operatore logico OR e l'operatore logico IN

- **OR:** Consente di confrontare campi diversi con condizioni diverse (es. Prezzo > 100 OR Categoria = 'Bikes').
- **IN:** È una scorciatoia sintattica dell'OR che si applica esclusivamente a un singolo campo confrontato con un elenco chiuso di valori specifici (es. Categoria IN ('Bikes', 'Clothing'), che equivale a scrivere Categoria = 'Bikes' OR Categoria = 'Clothing').

12. L'operatore logico BETWEEN include anche gli estremi del range specificato?

Sì, l'operatore BETWEEN in SQL è inclusivo. Scrivere Prezzo BETWEEN 10 AND 20 significa estrarre tutti i record dove il prezzo è maggiore o uguale a 10 e minore o uguale a 20 (≥ 10 e ≤ 20).



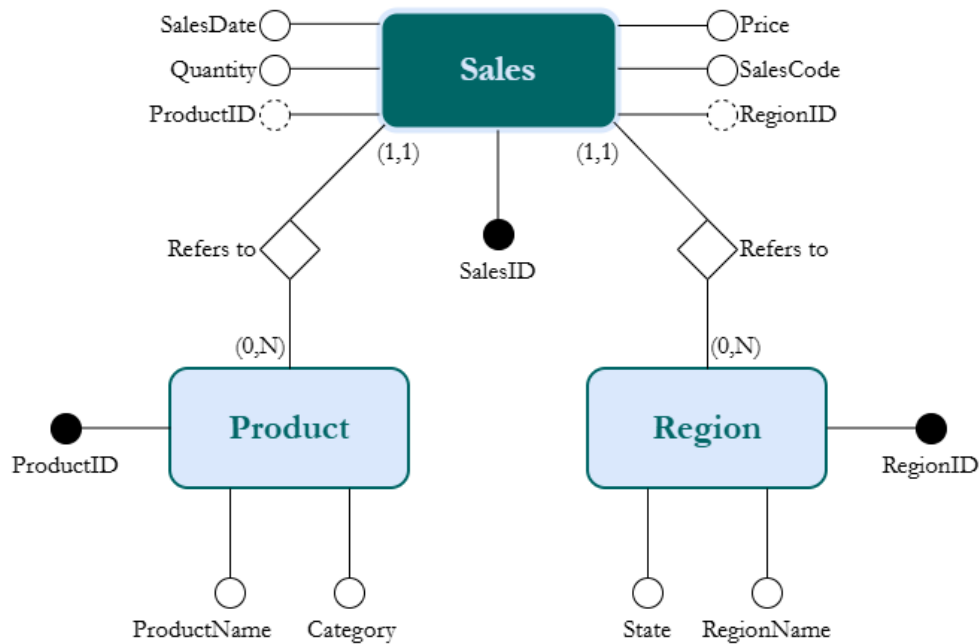
Risposte Case Study

Task 1: Proponi una progettazione concettuale e logica della base dati:

ToysGroup S.r.l.

Modello Concettuale

Diagramma Entità-Relazione (E/R)



ToysGroup S.r.l.

Modello Logico Relazionale

Dizionario dei Dati e Specifiche delle Tabelle

Product		
Chiave	Nome Colonna	Tipo di Dato e Vincolo
PK	ProductID	INT NOT NULL AUTO_INCREMENT
	ProductName	VARCHAR(100) NOT NULL
	Category	VARCHAR(50) NOT NULL

Region		
Chiave	Nome Colonna	Tipo di Dato e Vincolo
PK	RegionID	INT NOT NULL AUTO_INCREMENT
	RegionName	VARCHAR(100) NOT NULL
	State	VARCHAR(100) NOT NULL

Sales		
Chiave	Nome Colonna	Tipo di Dato e Vincolo
PK	SalesID	INT NOT NULL AUTO_INCREMENT
	SalesCode	VARCHAR(50) NOT NULL UNIQUE
	Quantity	INT NOT NULL
	Price	DECIMAL(18,2) NOT NULL
	SalesDate	DATE NOT NULL
FK	ProductID	INT NOT NULL
FK	RegionID	INT NOT NULL



Task 2: Descrivi la struttura delle tabelle che reputi utili e sufficienti a modellare lo scenario proposto tramite la sintassi DDL. Implementa fisicamente le tabelle utilizzando il DBMS SQL Server (o altro).

Per modellare lo scenario aziendale di *ToysGroup*, la base dati è stata strutturata su tre tabelle principali: due tabelle anagrafiche "forti" (Product e Region) e una tabella transazionale "debole" di associazione (Sales). Questa scomposizione garantisce il rispetto della Terza Forma Normale (3NF), minimizzando la ridondanza e preservando l'integrità referenziale.

Tabella: Product

Modella l'anagrafica dei giocattoli distribuiti dall'azienda. Include la gerarchia della categoria merceologica direttamente all'interno del record del prodotto.

- **ProductID:** Identificativo univoco del prodotto (Chiave Primaria, Auto-incrementale).
- **ProductName:** Nome commerciale dell'articolo (Dato testuale, obbligatorio).
- **Category:** Macro-categoria di appartenenza del giocattolo (Dato testuale, obbligatorio).

Tabella: Region

Modella la ripartizione geografica dei mercati di riferimento dell'azienda. Gestisce la gerarchia geografica accoppiando ciascuno Stato alla rispettiva area o regione globale di vendita.

- **RegionID:** Identificativo univoco della combinazione geografica (Chiave Primaria, Auto-incrementale).
- **RegionName:** Nome della macro-regione di vendita (es. WestEurope, SouthEurope) (Dato testuale, obbligatorio).
- **State:** Nome dello specifico Stato nazionale (es. France, Italy) (Dato testuale, obbligatorio).

Tabella: Sales

Tabella transazionale che registra ogni singola operazione di vendita effettuata. Agisce come tabella di collegamento tra le due anagrafiche principali.

- **SalesID:** Identificativo univoco della transazione (Chiave Primaria, Auto-incrementale).
- **SalesCode:** Codice alfanumerico del documento d'ordine (Dato testuale, obbligatorio).
- **Quantity:** Volume di pezzi venduti (Valore intero, obbligatorio).
- **Price:** Prezzo unitario di vendita applicato (Valore decimale monetario, obbligatorio).
- **SalesDate:** Data in cui è stata registrata la transazione (Tipo Date, obbligatorio).
- **ProductID:** Chiave Esterna collegata alla tabella Product (Integrità Referenziale).
- **RegionID:** Chiave Esterna collegata alla tabella Region (Integrità Referenziale).



TASK 2

Inizio con la creazione del Database e delle Entità

Elimino il database se già esistente per permettere esecuzioni ripetute

```
DROP DATABASE IF EXISTS ToysGroup;
```

Creo il database

```
CREATE DATABASE ToysGroup;
```

Seleziono il database

```
USE ToysGroup;
```

Tabella Product (Tabella Anagrafica Forte)

Contiene l'elenco dei prodotti e la loro categoria merceologica.
La PK è ProductID (auto-incrementale).

```
CREATE TABLE Product (
```

```
    ProductID          INT NOT NULL AUTO_INCREMENT PRIMARY KEY  
    ,ProductName        VARCHAR(100) NOT NULL  
    ,Category          VARCHAR(50) NOT NULL
```

```
);
```

Tabella Region (Tabella Anagrafica Forte)

Contiene gli stati e la regione geografica di appartenenza.
La PK è RegionID (auto-incrementale).

```
CREATE TABLE Region (
```

```
    RegionID          INT NOT NULL AUTO_INCREMENT PRIMARY KEY  
    ,RegionName        VARCHAR(100) NOT NULL  
    ,State            VARCHAR(100) NOT NULL
```

```
);
```

Tabella Sales (Tabella Debole / Transazionale)

Registra ogni vendita effettuata.



Contiene FK verso Product e Region.

```
CREATE TABLE Sales (  
    SalesID                INT NOT NULL AUTO_INCREMENT PRIMARY KEY  
    ,SalesCode              VARCHAR(50) NOT NULL UNIQUE  
    ,Quantity              INT NOT NULL  
    ,Price                 DECIMAL(18,2) NOT NULL  
    ,SalesDate             DATE NOT NULL  
    ,ProductID             INT NOT NULL  
    ,RegionID              INT NOT NULL  
    ,FOREIGN KEY (ProductID) REFERENCES Product(ProductID)  
    ,FOREIGN KEY (RegionID) REFERENCES Region(RegionID)  
);
```



Task 3: Popola le tabelle utilizzando dati a tua discrezione (sono sufficienti pochi record per tabella; riporta le query utilizzate).

=====

TASK 3

Popola le tabelle utilizzando dati a tua discrezione

=====

```
# -----  
# Popolamento tabella Product  
# -----
```

```
INSERT INTO Product  
(ProductName, Category) VALUES  
( 'Buzz Lightyear Action Figure', 'Action Figures'),  
( 'Sheriff Woody Doll', 'Dolls'),  
( 'Lightning McQueen Vehicle', 'Vehicles'),  
( 'Stitch Plush Toy', 'Plush Toys'),  
( 'Shrek Action Figure', 'Action Figures'),  
( 'Toothless Plush Toy', 'Plush Toys'),  
( 'Po Kung Fu Panda Doll', 'Dolls'),  
( 'Megamind Vehicle', 'Vehicles'),  
( 'Elsa Frozen Doll', 'Dolls'),  
( 'Alex the Lion Plush', 'Plush Toys');
```

```
# -----  
# Popolamento tabella Region  
# -----
```

```
INSERT INTO Region  
(RegionName, State) VALUES  
('WestEurope', 'France'),  
('WestEurope', 'Germany'),  
('SouthEurope', 'Italy'),  
('SouthEurope', 'Greece'),  
('NorthAmerica', 'United States'),  
('NorthAmerica', 'Canada'),  
('Asia', 'Japan'),  
('Asia', 'China');
```

```
# -----  
# Popolamento tabella Sales  
# -----
```

```
INSERT INTO Sales  
(SalesCode, Quantity, Price, SalesDate, ProductID, RegionID) VALUES  
('ORD-0001', 5, 25.00, '2023-03-10', 1, 1),  
('ORD-0002', 12, 15.50, '2023-06-15', 2, 3),  
('ORD-0003', 2, 45.00, '2023-09-22', 3, 5),  
('ORD-0004', 20, 10.00, '2023-12-05', 4, 7),  
('ORD-0005', 7, 22.00, '2024-01-18', 5, 2),  
('ORD-0006', 1, 35.00, '2024-04-22', 6, 4),  
('ORD-0007', 15, 18.00, '2024-07-30', 7, 6),  
('ORD-0008', 4, 50.00, '2024-11-12', 8, 8),  
('ORD-0009', 3, 25.00, '2025-02-14', 1, 3),  
('ORD-0010', 8, 15.50, '2025-04-19', 2, 6),  
('ORD-0011', 10, 45.00, '2025-06-25', 3, 1),  
('ORD-0012', 25, 10.00, '2025-08-05', 4, 5),  
('ORD-0013', 6, 22.00, '2025-09-12', 5, 8),  
('ORD-0014', 2, 35.00, '2025-10-20', 6, 2),  
('ORD-0015', 30, 18.00, '2025-11-05', 7, 4),  
('ORD-0016', 5, 50.00, '2025-11-28', 8, 7),  
('ORD-0017', 14, 25.00, '2025-12-10', 1, 2),  
('ORD-0018', 18, 15.50, '2025-12-18', 2, 4),  
('ORD-0019', 1, 45.00, '2025-12-23', 3, 6),  
('ORD-0020', 22, 10.00, '2025-12-30', 4, 8);
```



Task 4: Dopo aver popolate le tabelle, scrivi delle query utili a:

N.B. Nel corso delle query ho utilizzato l'alias talvolta con la parola chiave AS e talvolta senza. Questa scelta è intenzionale: in MySQL l'uso di AS è facoltativo, poiché l'alias viene comunque riconosciuto dal sistema anche quando la parola chiave non è esplicitata.

- 1) *Verificare che i campi definiti come PK siano univoci. In altre parole, scrivi una query per determinare l'univocità dei valori di ciascuna PK (una query per tabella implementata).*

Query 1

Verifica univocità delle Primary Key (PK)

Product

```
SELECT      ProductID
FROM        Product
GROUP BY    ProductID
HAVING      COUNT(*) > 1;
# --> 0 righe restituite = PK univoca
```

Ulteriore modo:

```
SELECT
    COUNT(*)                               ConteggioRighe
    ,COUNT(DISTINCT ProductID)            Conteggio
FROM Product;
```

Region

```
SELECT RegionID
FROM Region
GROUP BY RegionID
HAVING COUNT(*) > 1;
# --> 0 righe restituite = PK univoca
```

Sales

```
SELECT SalesID
FROM Sales
```

```
GROUP BY SalesID
HAVING COUNT(*) > 1;
# --> 0 righe restituite = PK univoca
```

```
# Ulteriore modo:
SELECT
    SalesID
    ,COUNT(*)           ConteggioRighe
FROM Sales
GROUP BY SalesID
HAVING ConteggioRighe > 1;
```



- 2) *Esporre l'elenco delle transazioni indicando nel result set il codice documento, la data, il nome del prodotto, la categoria del prodotto, il nome dello stato, il nome della regione di vendita e un campo booleano valorizzato in base alla condizione che siano passati più di 180 giorni dalla data vendita o meno (>180 -> True, <= 180 -> False).*

=====

Query 2

Elenco transazioni con indicatore temporale booleano (> 180 giorni)

=====

Mettiamo in relazione Sales con Product e Region tramite INNER JOIN.

La funzione DATEDIFF calcola la differenza in giorni tra la data odierna (CURDATE()) e la data della vendita. Se la differenza supera 180, il costrutto CASE restituisce 'True'.

```
SELECT
    S.SalesCode                AS CodiceDocumento
    ,S.SalesDate               AS DataVendita
    ,P.ProductName             AS NomeProdotto
    ,P.Category                AS CategoriaProdotto
    ,R.State                   AS NomeStato
    ,R.RegionName              AS NomeRegione
    ,CASE
        WHEN DATEDIFF(CURDATE(), S.SalesDate) > 180 THEN 'True'
        ELSE 'False'
    END                       AS Flag180gg
FROM Sales S
JOIN Product P               ON      S.ProductID = P.ProductID
JOIN Region R                ON      S.RegionID = R.RegionID;
```



- 3) *Esporre l'elenco dei prodotti che hanno venduto, in totale, una quantità maggiore della media delle vendite realizzate nell'ultimo anno censito. (ogni valore della condizione deve risultare da una query e non deve essere inserito a mano). Nel result set devono comparire solo il codice prodotto e il totale venduto.*

Query 3

Prodotti con quantità totale > media vendite dell'ultimo anno censito

Nel HAVING inseriamo una subquery che calcola la media (AVG) delle quantità vendute nell'ultimo anno censito. Quest'ultimo viene individuato dinamicamente con la subquery annidata MAX(YEAR(SalesDate)), che restituisce il 2025. Il filtro principale confronta la somma complessiva (storica) di ogni prodotto con tale parametro.

```
SELECT
    ProductID
    ,SUM(Quantity)          AS TotaleVenduto
FROM Sales
GROUP BY ProductID
HAVING SUM(Quantity) > (
    SELECT AVG(Quantity)
    FROM Sales
    WHERE YEAR(SalesDate) = (SELECT MAX(YEAR(SalesDate)) FROM Sales)
);
```

- 4) *Esporre l'elenco dei soli prodotti venduti e per ognuno di questi il fatturato totale per anno.*

Query 4

Prodotti venduti e relativo fatturato totale per anno

L'uso di INNER JOIN esclude automaticamente i prodotti che non hanno corrispondenze nella tabella Sales (i prodotti invenduti). Il fatturato è calcolato come prodotto tra prezzo e quantità venduta, aggregato tramite SUM per anno e per prodotto.

```
SELECT
```



```
P.ProductID
,P.ProductName
,YEAR(S.SalesDate)          AS Anno
,SUM(S.Price * S.Quantity)  AS FatturatoTotale
FROM Sales S
JOIN Product P ON S.ProductID = P.ProductID
GROUP BY P.ProductID, P.ProductName, YEAR(S.SalesDate)
ORDER BY P.ProductID, Anno;
```

5) *Esporre il fatturato totale per stato per anno. Ordina il risultato per data e per fatturato decrescente.*

Query 5

Fatturato totale per stato e per anno

Aggreghiamo i dati di vendita estraendo l'anno dalla data e associandoli allo Stato di riferimento ricavato dalla tabella geografica Region. L'ordinamento è cronologico crescente (Anno ASC) e monetario decrescente (FatturatoTotale DESC).

```
SELECT
  R.State          AS Stato
  ,YEAR(S.SalesDate) AS Anno
  ,SUM(S.Price * S.Quantity) AS FatturatoTotale
FROM Sales S
JOIN Region R ON S.RegionID = R.RegionID
GROUP BY R.State, YEAR(S.SalesDate)
ORDER BY Anno ASC, FatturatoTotale DESC;
```

6) *Rispondere alla seguente domanda: qual è la categoria di articoli maggiormente richiesta dal mercato?*

Query 6

Categoria di articoli maggiormente richiesta dal mercato

Raggruppiamo i dati per categoria merceologica e calcoliamo il volume totale dei pezzi venduti tramite SUM(Quantity). Ordinando i risultati in modo decrescente e applicando LIMIT 1, soliamo il record della categoria più popolare.



```
SELECT
    P.Category                AS Categoria
    ,SUM(S.Quantity)          AS TotaleQuantitaVenduta
FROM Sales S
JOIN Product P ON S.ProductID = P.ProductID
GROUP BY P.Category
ORDER BY TotaleQuantitaVenduta DESC
LIMIT 1;
```

7) *Rispondere alla seguente domanda: quali sono i prodotti invenduti? Proponi due approcci risolutivi differenti.*

Query 7

Identificazione dei prodotti invenduti (Due approcci risolutivi)

APPROCCIO 1

LEFT JOIN con clausola WHERE su chiave esterna IS NULL

Estraiamo TUTTI i prodotti presenti nell'anagrafica. Se un prodotto non ha vendite, i campi della tabella Sales assumeranno valore NULL. Il filtro isola esattamente questi casi.

```
SELECT
    P.ProductID
    ,P.ProductName
    ,P.Category
FROM Product P
LEFT JOIN Sales S ON P.ProductID = S.ProductID
WHERE S.SalesID IS NULL;
```

APPROCCIO 2

Subquery con operatore logico NOT IN

La subquery genera un elenco univoco di tutti i ProductID associati a transazioni reali. La query principale seleziona i prodotti i cui ID non compaiono in quell'elenco.

```
SELECT
    P.ProductID
    ,P.ProductName
```



```
,P.Category  
FROM Product P  
WHERE P.ProductID NOT IN (  
    SELECT ProductID FROM Sales  
);
```

8) Creare una vista sui prodotti in modo tale da esporre una “versione denormalizzata” delle informazioni utili (codice prodotto, nome prodotto, nome categoria).

=====

Query 8
Creazione Vista Prodotti Denormalizzata

=====

```
CREATE OR REPLACE VIEW ProdottiDenormalizzati AS (  
    SELECT  
        ProductID           CodiceProdotto  
        ,ProductName        NomeProdotto  
        ,Category           NomeCategoria  
    FROM Product  
);
```

9) Creare una vista per le informazioni geografiche.

=====

Query 9 - Creazione Vista Informazioni Geografiche

=====

```
CREATE OR REPLACE VIEW InformazioniGeografiche AS (  
    SELECT  
        RegionID  
        ,RegionName        NomeRegione  
        ,State             NomeStato  
    FROM Region  
);
```

Fine